

MULTIPLE CLUSTERING:

From Classical Foundations to Interactive, User-Guided
Methods



Presenters



Jian Pei

Arthur S. Pearce Distinguished Professor

Department of Computer Science, Duke University

j.pei@duke.edu



Juhua Hu

Associate Professor

School of Engineering and Technology,
University of Washington

juhuah@uw.edu



Jiawei Yao

Ph.D. Student

School of Engineering and Technology,
University of Washington

jwyao@uw.edu

Tutorial Overview

Understand what *multiple clustering* means beyond a single “best” partition

- Formal problem setup and the quality–diversity trade-off

Survey the major families of methods

- Classical, deep representation-based, and multi-view / multi-source approaches

Connect algorithms to practice

- How to pick methods for your data and how to interpret the resulting clusterings

Look ahead to user-guided and natural-language–driven multiple clustering

- Towards interactive exploration aligned with human intent

Roadmap

Part I — Motivation & Problem Formulation

- Why multiple clusterings? Formal definitions & examples

Part II — Classical Foundations

- Alternative clusterings, subspace and spectral methods

Part III — Deep Representation–Based Multiple Clustering

- Autoencoders, disentangled representations, contrastive & adversarial training

Part IV — Multi-View / Multi-Source Multiple Clustering

- Combining heterogeneous views and modalities

Part V — User-Guided & Interactive Methods

- From feature weights to natural-language interfaces (e.g., CLIP + LLMs)

Part VI — Evaluation & Open Problems

- Metrics, benchmarks, and emerging research directions

Motivation & Problem Formulation

Single Clustering

Single Clustering

- Setting
 - Data points $X = \{x_1, x_2, \dots, x_N\}$
 - One partition $C = \{C_1, C_2, \dots, C_K\}$ with $C_k \subseteq X$
- Typical objective
 - High intra-cluster similarity, low inter-cluster similarity
 - Example (k-means):

$$\min_{C, \{\mu_k\}} \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

- Classical algorithms
 - Centroid-based: k-means, k-medoids
 - Graph-based: spectral clustering
 - Density-based: DBSCAN, OPTICS

Why a Single Partition Is Not Enough

Implicit “one-partition-only” assumption

- The algorithm returns a *single* clustering that is treated as the “right” view of the data

Hidden ambiguity in real datasets

- Different normalizations, feature subsets, or distance metrics can all lead to different clusterings
- Each of these choices implicitly selects *one* semantic perspective and hides the others

Bridge to multiple clustering

- Instead of committing to one partition, we explicitly search for *several* high-quality, non-redundant clusterings
- Each partition provides a valid explanation of the same data from a different viewpoint

One Dataset, Many Valid Clusterings — A Toy Example

Let the dataset be $X = \{x_1, \dots, x_N\}$

Example 1: clustering by shape

$$C^{(\text{shape})} = \{C_{\text{circle}}, C_{\text{square}}, C_{\text{triangle}}\},$$

where each cluster groups points with the same shape.

Example 2: clustering by color

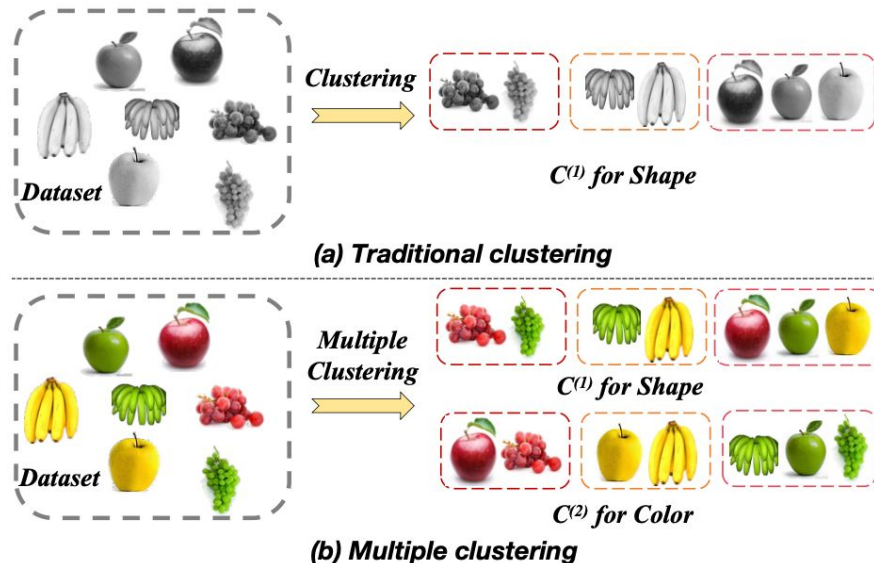
$$C^{(\text{color})} = \{C_{\text{red}}, C_{\text{green}}, C_{\text{blue}}\},$$

where each cluster groups points with the same color.

Both

$$C^{(\text{shape})} \quad \text{and} \quad C^{(\text{color})}$$

are high-quality clusterings of the same dataset, but they answer different questions.



Explicitly enumerate several high-quality, semantically different partitions of the same data.

Formalizing Multiple Clusterings — Quality vs. Diversity

Problem setting

- Data: $X = \{x_1, x_2, \dots, x_N\}$
- Multiple clusterings instead of one
 - We aim to find M clustering solutions

$$\mathcal{C} = \{C^{(1)}, C^{(2)}, \dots, C^{(M)}\}$$

- Each solution is a partition

- For solution p ,

$$C^{(p)} = \{C_1^{(p)}, \dots, C_{K_p}^{(p)}\}$$

- where clusters in $C^{(p)}$
 - do not overlap: $C_k^{(p)} \cap C_{k'}^{(p)} = \emptyset$ for $k \neq k'$,
 - and cover all points: $\bigcup_k C_k^{(p)} = X$

Formalizing Multiple Clusterings — Quality vs. Diversity

Quality–diversity view

- Quality of a single clustering
 - $Q(C^{(p)})$ measures how good solution p is by itself
 - Examples: intra-cluster compactness, inter-cluster separation, NMI or ARI to latent labels
- Diversity across solutions
 - $\text{Sim}(C^{(p)}, C^{(q)})$ measures similarity between two solutions
 - e.g., normalized mutual information, variation of information, or cluster assignment overlap
- A generic objective

$$\max_{C^{(1)}, \dots, C^{(M)}} \sum_{p=1}^M Q(C^{(p)}) - \lambda \sum_{p < q} \text{Sim}(C^{(p)}, C^{(q)}),$$

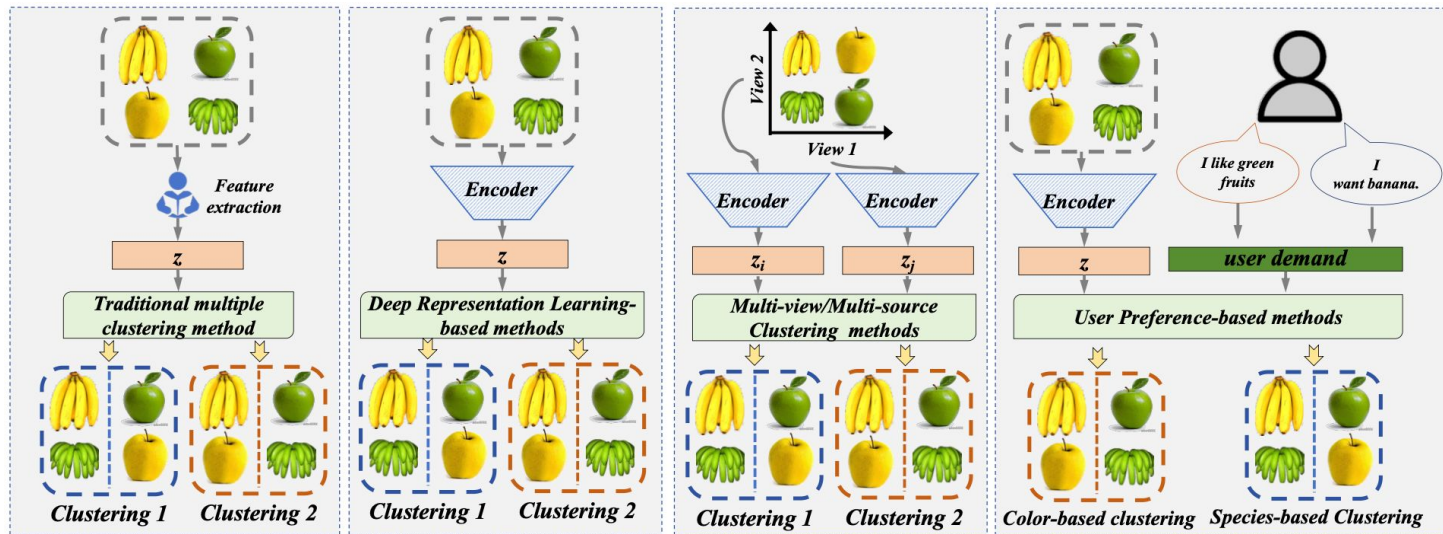
where $\lambda > 0$ controls how strongly we penalize redundancy between solutions.

*Multiple clustering is a **multi-objective** problem: each solution to be a good clustering and the set of solutions to be diverse, not just copies of each other.*

A Taxonomy of Multiple Clustering Methods

Multiple clustering methods can be roughly organized along three major families:

1. Classical / non-deep methods (Part II)
2. Deep representation-based methods (Part III)
3. Multi-view / multi-source and user-guided methods (Part IV and Part V)



Classical Multiple Clustering Foundations

Classical Multiple Clustering Foundations

What we focus on in this part

- How to obtain *multiple* clusterings using classical (non-deep) tools
 - Alternative and disparate clusterings
 - Subspace-based multiple clustering
 - Multiple spectral and density-based clusterings
- How to formalize “non-redundant” solutions in classical setups
 - Modify the objective to penalize similarity between solutions
 - Or search for distinct subspaces and views of the same data

Key questions

- Given one reasonable clustering, how can we find **another** clustering that is
 - still high-quality, but
 - clearly different in structure or semantics?
- How can **feature subspaces**, transformations, or graph structures be used to reveal alternative partitions?
- What do we gain and lose with classical methods compared to deep representation-based approaches?
 - Interpretation vs. flexibility
 - Optimization simplicity vs. expressive power

Alternative / Disparate Clusterings: Basic Idea

Basic idea

- Start from **one** reasonable clustering solution $C^{(1)}$
- Seek a **second** solution $C^{(2)}$ that
 - is still a high-quality clustering by itself. and
 - is **as different as possible** from $C^{(1)}$
- Generalize to multiple solutions $C^{(1)}, C^{(2)}, \dots, C^{(M)}$
- This line of work is often called *alternative clustering* or *disparate clustering*.

A generic classical objective

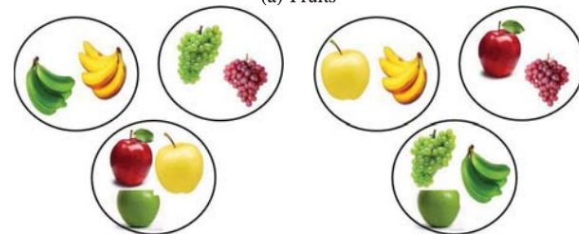
- Let $Q(C)$ be the quality of a clustering (e.g., negative k-means distortion, graph cut quality).
- Let $\text{Sim}(C^{(p)}, C^{(q)})$ measure similarity between two clusterings (e.g., normalized mutual information, label agreement, or shared edges).
- For two solutions: where $\lambda > 0$ controls how strongly we penalize redundancy.

$$\max_{C^{(1)}, C^{(2)}} Q(C^{(1)}) + Q(C^{(2)}) - \lambda \text{Sim}(C^{(1)}, C^{(2)}),$$

- For multiple solutions:
$$\max_{C^{(1)}, \dots, C^{(M)}} \sum_{p=1}^M Q(C^{(p)}) - \lambda \sum_{p < q} \text{Sim}(C^{(p)}, C^{(q)})$$



(a) Fruits



(b) Clustering by species

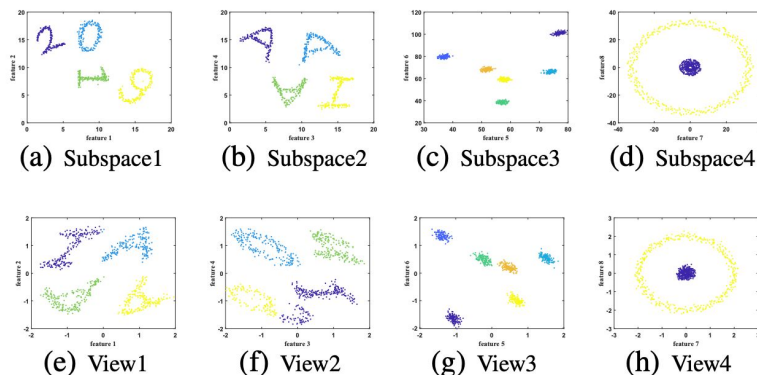
(c) Clustering by color

“Example of alternative clusterings: the same fruits clustered by species vs. by color (Hu et al., ICDM 2015).”

Subspace-Based Multiple Clusterings

Motivation

- Real-world data are often **high-dimensional**.
 - Different cluster structures may live in **different subsets of features**.
 - Example: patients clustered by demographics vs. by lab tests; images clustered by color vs. by texture.
- A single clustering in the **full feature space** may mix these structures and hide interesting patterns.
 - Different cluster structures may live in **different subsets of features**.
 - Example: patients clustered by demographics vs. by lab tests; images clustered by color vs. by texture.
- Idea:
 - Find **multiple subspaces** of the original feature space, and perform clustering **within each subspace**.
 - Each subspace → one meaningful clustering solution.



Four different clusterings in four subspaces (a-d), and the four clusterings explored by MISC (e-h).

Subspace-Based Multiple Clusterings

Basic formulation

- Let $X \in \mathbb{R}^{N \times D}$ be the data matrix.
- For each solution $p = 1, \dots, M$:
 - Choose a subspace $S^{(p)}$ (e.g., a subset of features, or a linear projection).
 - Compute projected features

$$z_i^{(p)} = W^{(p)\top} x_i, \quad i = 1, \dots, N,$$

where $W^{(p)}$ defines subspace $S^{(p)}$.

- Run a standard clustering algorithm (e.g., k-means, spectral) on $\{z_i^{(p)}\}$ to get clustering $C^{(p)}$
- We want:
 - Each $(S^{(p)}, C^{(p)})$ to give a **high-quality** clustering in its own subspace.
 - The subspaces $\{S^{(p)}\}$ to be as **independent / non-overlapping** as possible (e.g., via mutual information or MDL).
- Generic objective sketch:

$$\max_{\{W^{(p)}, C^{(p)}\}} \sum_{p=1}^M Q(C^{(p)}, W^{(p)}) - \gamma R_{\text{subspace}}(\{W^{(p)}\}),$$

Where Q is clustering quality in each subspace and R_{subspace} penalizes dependence / overlap between subspaces.

*Subspace-based methods explain **diverse clusterings** by associating each solution with its **own low-dimensional subspace**, making the structure behind each clustering more interpretable.*

Case Study — Multiple Independent Subspace Clusterings (MISC, AAAI 2019)

Key idea

- Discover several **independent subspaces** in the original feature space.
- In each subspace, run a standard clustering algorithm.
- Output: a set of clusterings and each tied to its own subspace and thus naturally **non-redundant**.

Two-stage procedure

- Stage I — Independent subspace discovery
 - Input data matrix $X \in \mathbb{R}^{N \times D}$
 - Use independent subspace analysis to decompose the feature space into candidate subspaces

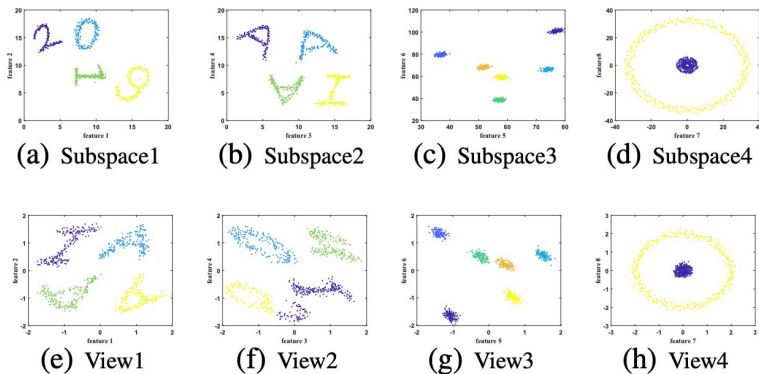
$$S^{(1)}, S^{(2)}, \dots, S^{(M)}$$

- Encourage **statistical independence** between subspaces, and use a **minimum description length (MDL)** criterion to decide
 - how many subspaces to keep, and
 - which ones are meaningful.

Case Study — Multiple Independent Subspace Clusterings (MISC, AAAI 2019)

Two-stage procedure

- Stage II — Clustering within each subspace
 - For each selected subspace $S^{(p)}$
 - Project data: $z_i^{(p)} = W^{(p)\top} x_i$
 - Build a similarity graph on $\{z_i^{(p)}\}$
 - Apply graph-regularized (kernel) semi-NMF to obtain cluster memberships and derive a hard clustering $C^{(p)}$



MISC discovers multiple independent subspaces, each with its own clustering (Wang et al., AAAI 2019)

More Classical Families: Spectral Multiple Clusterings

Spectral / graph-based approaches

- Start from a similarity graph over the data (affinity matrix).
- A **standard spectral clustering** uses one set of eigenvectors to produce **one** partition.
- To obtain **multiple** clusterings, classical methods modify the graph or the embedding, e.g.:
 - Reweight or remove edges that strongly support an existing clustering, so that the next solution is forced to rely on **different connections** in the graph.
 - Use different combinations of eigenvectors or impose **independence / orthogonality** between the embeddings associated with different solutions.

- Intuition:

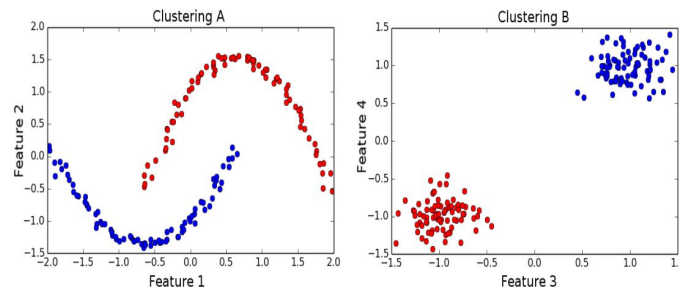
Each clustering corresponds to a different way of cutting or embedding the **same graph**, highlighting different structures or communities.

More Classical Families: Density-Based Multiple Clusterings

Density-based / multi-resolution clusterings

- Density-based methods (DBSCAN, OPTICS, HDBSCAN, etc.) naturally support **multi-resolution** views:
 - Changing the density scale (e.g., epsilon, minPts) merges or splits clusters.
 - Different scales correspond to **coarse** vs. **fine** clusterings of the same data.
- For multiple clusterings, we can **explicitly keep several scales** instead of picking just one:
 - A coarse partition showing a few large groups.
 - A fine partition revealing subclusters inside each group.
- Intuition:

Diversity comes from **granularity**:
the same structure is viewed at different density / resolution levels.



Example of alternative clusterings on a 4D dataset: different projections reveal different clusters (Wu et al., AISTATS 2018).

From Classical to Deep Multiple Clusterings

What we learned from classical methods

- Alternative / disparate clusterings
 - Add a *diversity penalty* so new solutions are high-quality but not redundant.
- Subspace-based methods (e.g., MISC)
 - Discover **multiple subspaces**, cluster in each subspace → different views of the same data.
- Spectral & density-based approaches
 - Explore **different graph embeddings / cuts** or **different density scales** (coarse vs. fine).

Where classical methods fall short

- Rely on **fixed or shallow features** → struggle with complex image, text, bio, and multi-modal data.
- Mostly designed for **single-view** settings → limited support for multi-view / multi-source inputs.
- User control is indirect (feature weights, parameters) → hard to align clusterings with human intent.

Natural next step: deep representations

- Learn **latent spaces** directly from raw data (autoencoders, graph/tensor encoders, contrastive encoders).
- Use **multiple heads / branches / subspaces** in one network to realize multiple clusterings.
- Provide hooks for **multi-view integration** and **user-guided clustering** (proxies, prompts, preferences).

Deep Representation–based Multiple Clustering

Deep Representation-Based Multiple Clustering

Why go deep?

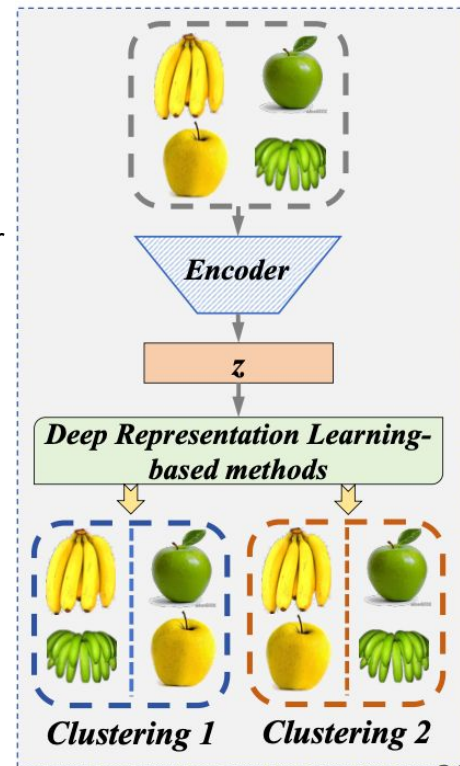
- Classical methods work in the **original feature space** or hand-crafted subspaces.
- Deep methods first learn a **latent representation**

$$z_i = E_{\theta}(x_i),$$

- Where E_{θ} is a deep encoder (autoencoder, GNN, etc.).
- Multiple clusterings are then realized in **latent space**, which can disentangle complex, non-linear structure.

Core architecture

- **Encoder**
 - Map input data x_i (images, text, multi-modal inputs) to latent vectors z_i .
- **Multiple clustering heads / subspaces**
 - Head 1: clustering $C^{(1)}$ on $\{z_i^{(1)}\}$
 - Head 2: clustering $C^{(2)}$ on $\{z_i^{(2)}\}$
 - ... each head or subspace corresponds to one clustering view.
- **Training objectives**
 - Reconstruction loss to preserve information.
 - Clustering loss to sharpen assignments.
 - Diversity regularizers so different heads are not redundant.



Design Space of Deep Multiple Clustering Methods

Backbone representation

- Deep encoder E_θ maps input x_i to latent vector $z_i = E_\theta(x_i)$
- Choices of backbone:
 - Autoencoders / VAEs (images, tabular, text).
 - Graph / tensor encoders (relational or multi-relational data).
 - Contrastive encoders with data augmentations.

How multiple clusterings are realized

- **Multi-head on a shared latent space**
 - One encoder, multiple clustering heads: $z_i \longrightarrow \{h^{(1)}(z_i), h^{(2)}(z_i), \dots, h^{(M)}(z_i)\}$.
 - One encoder, multiple clustering heads: $C^{(p)}$
- **Latent subspaces / factorization**
 - Split z_i into components

$$z_i = [z_i^{(1)} \parallel z_i^{(2)} \parallel \dots \parallel z_i^{(M)}],$$

or learn multiple projection matrices $W^{(p)}$

- Cluster each component / subspace to obtain different views.

Design Space of Deep Multiple Clustering Methods

Training objective: quality + diversity:

- Typical overall loss:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \sum_{p=1}^M \mathcal{L}_{\text{clust}}^{(p)} + \lambda \mathcal{L}_{\text{div}},$$

where

- $\mathcal{L}_{\text{rec}}$: reconstruction / self-supervised loss,
- $\mathcal{L}_{\text{clust}}^{(p)}$: clustering loss for head ppp (e.g., DEC-style, k-means style),
- $\mathcal{L}_{\text{div}}$: encourages solutions to be non-redundant (penalize similarity between heads or subspaces).
- Many methods mainly differ in **how they design** E_{θ} and **how they implement** $\mathcal{L}_{\text{div}}$ (collaborative, contrastive, adversarial, etc.).

*Deep methods combine a learned latent backbone with **multiple heads / subspaces** and an explicit **quality + diversity** objective.*

Autoencoder-Based Deep Multiple Clustering

Basic idea

- Use a deep autoencoder to learn a latent representation

$$z_i = E_\theta(x_i), \quad \hat{x}_i = D_\phi(z_i).$$

- Attach **multiple clustering heads** on top of the latent space:

$$z_i \longrightarrow h^{(1)}(z_i), h^{(2)}(z_i), \dots, h^{(M)}(z_i).$$

- Each head $h^{(p)}$ produces one clustering solution $C^{(p)}$ on the same data.

Typical training objective

- **Reconstruction loss** (autoencoder)
- **Clustering loss for each head**: DEC-style objective: encourage confident, compact clusters in latent space

$$\mathcal{L}_{\text{clust}}^{(p)} = \text{KL}(P^{(p)} \parallel Q^{(p)}),$$

where $Q^{(p)}$ is the soft assignment of z_i to clusters of head p ,

And $P^{(p)}$ is a sharpened target distribution

- **Diversity regularizer across heads**: Penalize similarity between assignments or centroids, e.g.

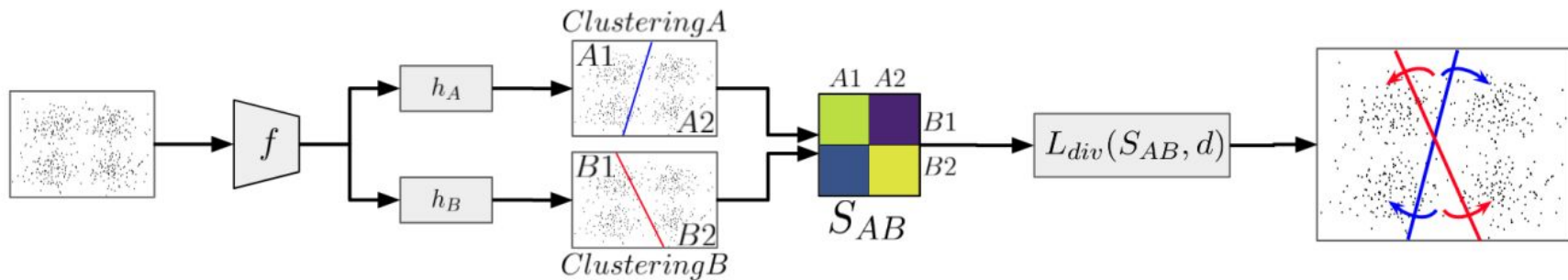
$$\mathcal{L}_{\text{div}} = \sum_{p < q} \text{Sim}(C^{(p)}, C^{(q)}) \quad (\text{NMI, cosine between centroids, etc.}).$$

- **Overall loss**: $\mathcal{L} = \mathcal{L}_{\text{rec}} + \sum_{p=1}^M \mathcal{L}_{\text{clust}}^{(p)} + \lambda \mathcal{L}_{\text{div}}.$

Autoencoder-Based Deep Multiple Clustering

Many deep multiple clustering models follow this template:

- ENRC — AE + clustering objective for alternative clusterings.
- iMClusters — AE + multi-head attention to get **diverse salient embeddings**.
- AugDMC — AE / CNN backbone + augmentations to specialize different heads



Overview of a deep multiple-clustering framework with a shared encoder and multiple heads regularized by a diversity loss (Metaxas et al., CVPR 2023).

Case Study — Dual-Disentangled Deep Multiple Clustering (DDMC)

Motivation

- Many deep multiple clustering methods
 - Learn several feature embeddings with limited overlap,
 - Then run k-means on each embedding.
- Limitations
 - Feature diversity is only **indirectly** related to clustering diversity.
 - Clustering objective is often **not involved** when learning representations.
- DDMC idea
Learn **disentangled latent factors** that correspond to different aspects of data, and tie them **directly** to clustering.

Case Study — Dual-Disentangled Deep Multiple Clustering (DDMC)

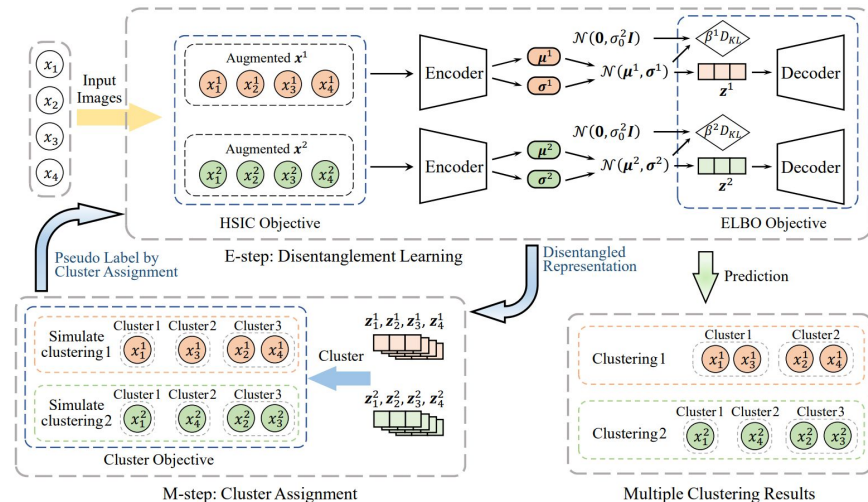
High-level idea: dual disentanglement

- Input image x_i is mapped to several latent factors

$$z_i^{(1)}, z_i^{(2)}, \dots, z_i^{(L)},$$

each intended to capture a different *aspect* (e.g., color, shape, style).

- Coarse-grained disentanglement**
 - Use multiple data augmentations and independence constraints to push different branches to focus on **different coarse aspects**.
- Fine-grained disentanglement**
 - Within each aspect, learn a compact latent code and discrete assignment that are suitable for clustering.



DDMC — Training Procedure and Benefits

Variational EM training (overview)

- DDMC uses an **EM-style** training loop:

E-step — learn disentangled representations

- Optimize encoder and latent variables $\{z_i^{(\ell)}\}$
to improve both coarse- and fine-grained disentanglement.
- Coarse level: encourage **independence** between augmented views / aspects.
- Fine level: VAE-style ELBO enforces structured latent factors for each aspect.

- **M-step — optimize cluster assignments**

- With representations fixed, update cluster centers U and assignments y_i .
- A k-means-style objective in disentangled space, e.g. $\mathcal{L}_{\text{clust}} = \sum_i \|z_i - U y_i\|^2$,
refined iteratively to improve cluster quality.

Summary — Deep Multiple Clustering

What we covered

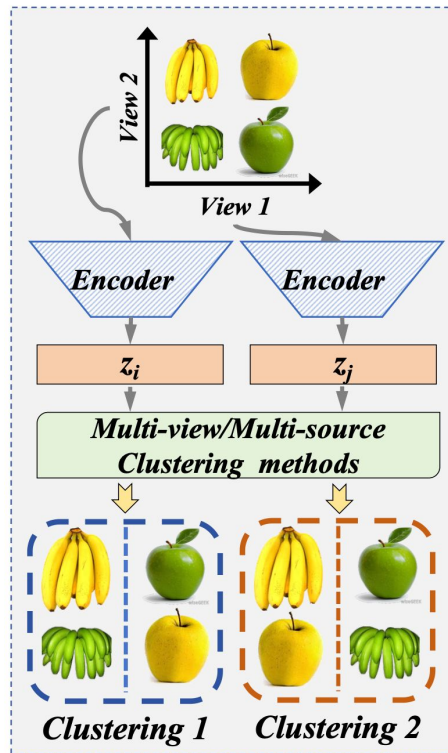
- Deep encoders learn a **latent space** $z = E_{\theta}(x)$ from raw data.
- Multiple clusterings are realized by
 - **multi-head** classifiers on a shared latent space, and/or
 - **multiple latent subspaces / factors**.
- Training combines
 - reconstruction / self-supervised loss,
 - clustering loss for each head,
 - a **diversity loss** between heads (e.g., DivClust).
- DDMC shows how **disentangled factors + EM training** can align different clusterings with different semantic aspects.

Deep-specific insights

- Diversity is no longer just “different feature subsets”, but can be grounded in **different latent factors** (color, shape, style, etc.).
- Clustering quality and diversity can be optimized **end-to-end** with the representation.
- Deep frameworks make it easier to plug in **multi-view inputs** and **user guidance** in later sections.

What Comes Next — Multi-View / Multi-Source Multiple Clustering

- Many real datasets are **multi-view / multi-source** by nature
 - e.g., image + text, multi-omics, multi-sensor data
- In **next part**, we will see how deep multiple clustering frameworks
 - extend to multi-view and multi-source data, and
 - balance **shared** vs. **view-specific** structure across views.



Multi-View / Multi-Source Multiple Clustering

Multi-View / Multi-Source Multiple Clustering

Why multi-view?

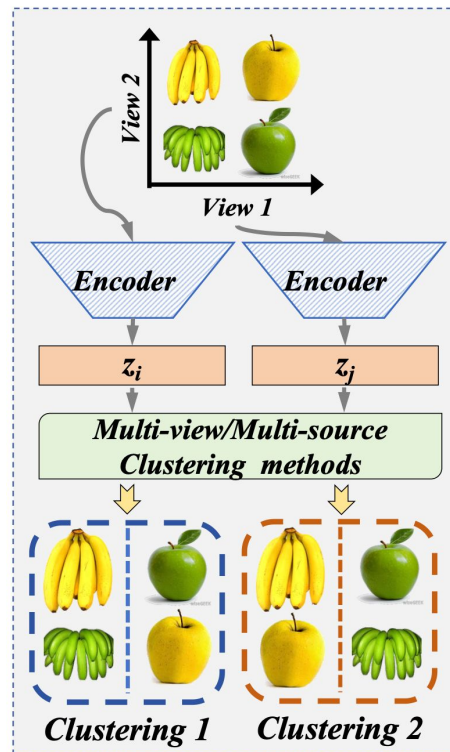
- Many objects have **multiple views**:
 - Image + text, audio + video, multi-omics, multi-sensor data
- Each view captures **different aspects** of the same entity
 - Some structure is **shared**, some is **view-specific**

What we focus on in this part

- Deep multi-view encoders for multiple clustering
 - Shared latent space vs. view-specific latent spaces
- Balancing **consensus** and **complementarity**
 - When should views agree on a clustering?
 - When should they intentionally disagree to reveal new structure?
- Handling realistic issues
 - Incomplete / noisy views
 - Scalability to large, heterogeneous datasets

Running theme

- How to design multi-view models that
 - keep the **quality** of each clustering, and
 - use **cross-view structure** to enhance diversity, robustness, and interpretability.



Multi-View Data: Setup and Examples

Multi-view setting

- We observe V views of the **same set of objects**:

$$X^{(1)}, X^{(2)}, \dots, X^{(V)}, \quad X^{(v)} = \{x_1^{(v)}, \dots, x_N^{(v)}\}.$$

- Each object i has one sample in each view: $(x_i^{(1)}, x_i^{(2)}, \dots, x_i^{(V)})$

are different views of the same underlying entity.

Examples

- Vision–language:
 - $v = 1$: image, $v = 2$: text (caption, tags, metadata)
- Multi-omics:
 - $v = 1$: gene expression, $v = 2$: chromatin accessibility, $v = 3$: methylation
- Multi-sensor:
 - Different sensors measuring the same scene or event

Shared vs. View-Specific Structure in Multi-View Multiple Clustering

Encoders and latent spaces

- Each view has its own encoder:

$$z_i^{(v)} = E_{\theta}^{(v)}(x_i^{(v)}).$$

- Latent structure can be decomposed into

- **Shared part** (common across views):

$$z_i^{(\text{sh})}$$

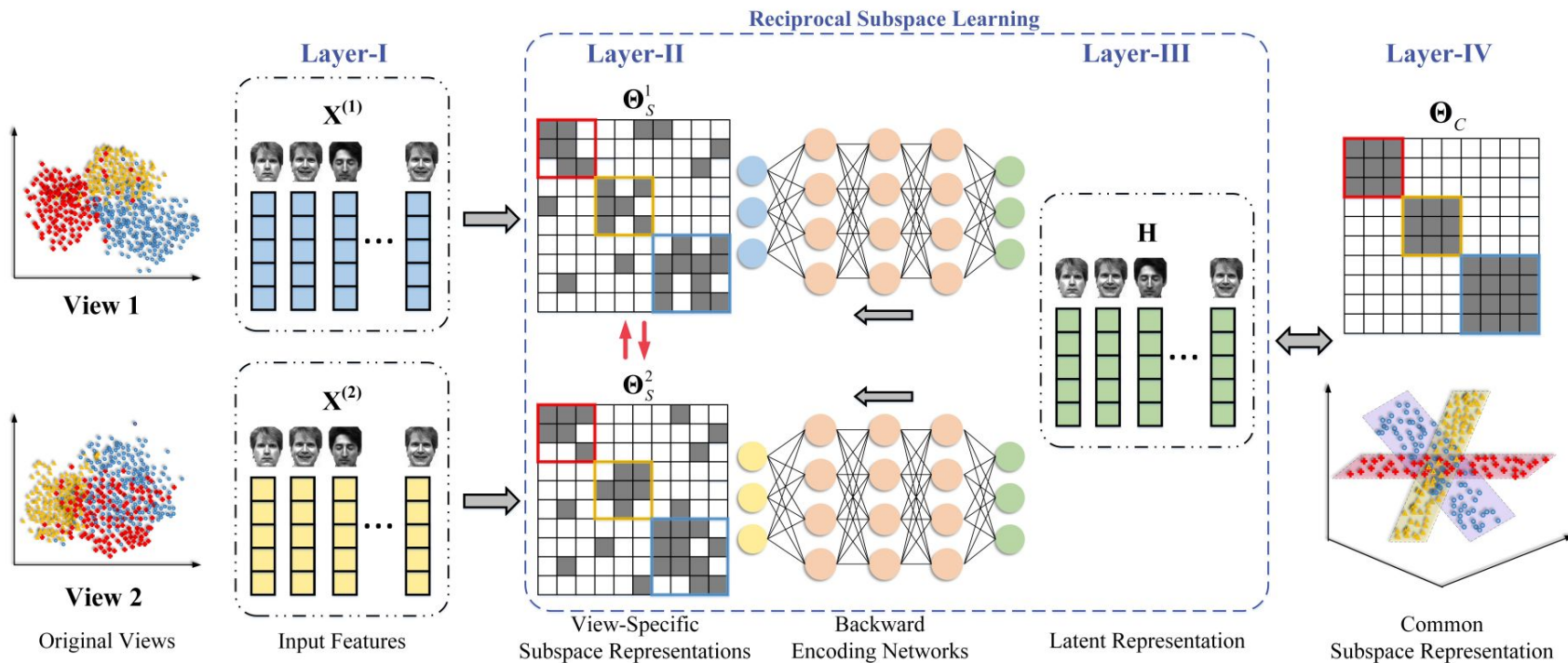
- **View-specific parts** (unique to each view):

$$z_i^{(v,\text{sp})}.$$

- Idea:

Use $z_i^{(\text{sh})}$ for **consensus** clustering,
and $z_i^{(v,\text{sp})}$ to capture **complementary** view-specific clusterings.

Shared vs. View-Specific Structure in Multi-View Multiple Clustering



Complementarity in Multi-View Subspace Clustering (DiMSC, CVPR 2015)

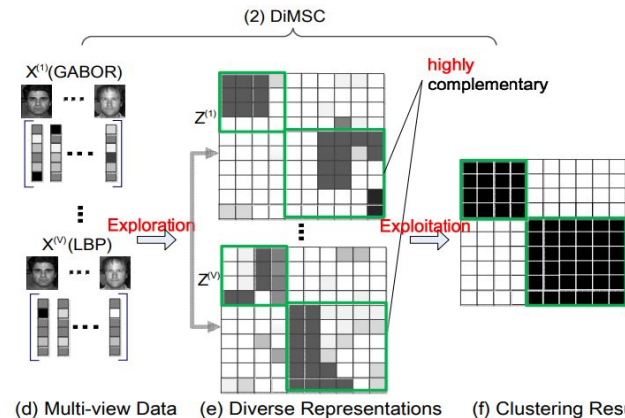
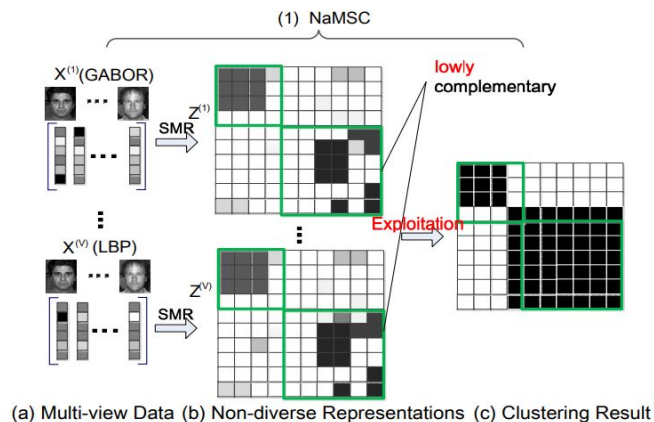
Motivation

- In multi-view clustering, we do **not** just want views to agree.
- We also care about **complementary information**:
 - Each view should contribute something *new* about the structure.
- If we only build subspace / similarity matrices **independently** and then combine them, we may lose this complementarity.

Complementarity in Multi-View Subspace Clustering (DiMSC, CVPR 2015)

DiMSC: Diversity-Induced Multi-View Subspace Clustering

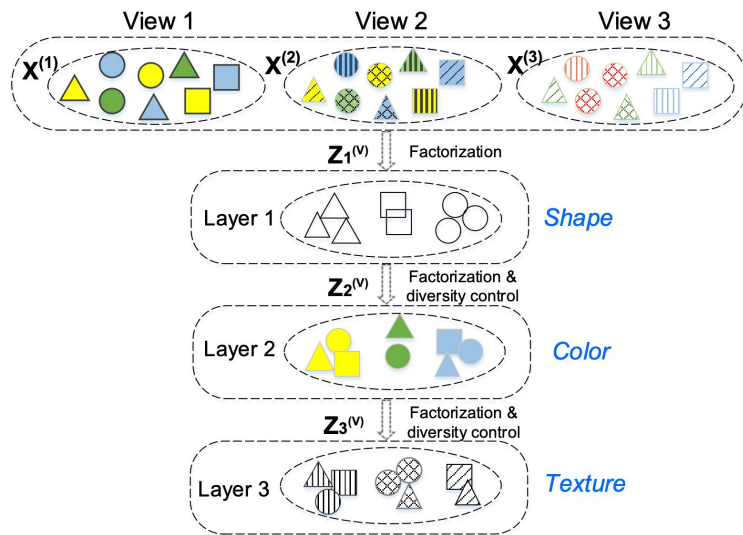
- Start from self-representation for each view:
 - For view v , learn a subspace representation $Z^{(v)}$ and similarity matrix $S^{(v)}$.
- Naive approach (NaMSC):**
 - Optimize each $Z^{(v)}$ independently and then combine similarities.
 - No guarantee that the views are really complementary.
- DiMSC adds a diversity term** across views:
 - Uses HSIC (Hilbert–Schmidt Independence Criterion) to penalize **dependence** between subspace representations $\{Z^{(v)}\}$.
 - Intuition: Encourage the learned representations of different views to be sufficiently *diverse*, so they capture complementary structure.
- Final clustering is obtained by spectral clustering on a similarity matrix built from all diversified subspace representations.



Case Study — DMClusts: Deep Multi-View Multiple Clusterings

Intuition

- Multi-view data can support **several** meaningful clusterings
 - e.g., group objects by *shape*, *color*, or *texture* across views.
- Most multi-view methods output **one** consensus clustering.
- DMClusts asks:
Can we use deep matrix factorization to generate multiple high-quality, low-redundancy clusterings from multi-view data?



DMClusts: one clustering per layer, with different views weighted for shape, color, and texture (Wei et al., AAAI 2020).

Case Study — DMClusts: Deep Multi-View Multiple Clusterings

Deep matrix factorization + multiple clusterings

- For each view v , factorize the data with multiple layers:

$$X^{(v)} \approx W_1^{(v)} W_2^{(v)} \dots W_L^{(v)} H,$$

where each layer ℓ produces a **latent feature matrix** H_ℓ .

- **One clustering per layer**
 - On each H_ℓ , run a clustering algorithm (e.g., k-means / semi-NMF) to obtain clustering $C^{(\ell)}$.
 - Result: L different clusterings $C^{(1)}, \dots, C^{(L)}$ from the same multi-view data.
- **View-weighted fusion**
 - For each layer ℓ , learn view weights $\alpha_{v,\ell}$:

$$\tilde{H}_\ell = \sum_{v=1}^V \alpha_{v,\ell} H_\ell^{(v)},$$

so each clustering can emphasize the views that matter most (e.g., shape uses all views; color uses view 1 & 2; texture uses view 2 & 3).

Case Study — DMClusts: Deep Multi-View Multiple Clusterings

Redundancy control (sketch)

- DMClusts builds a **co-association matrix** $A^{(\ell)}$ from each clustering, measuring how often pairs of points are grouped together.
- A balanced redundancy term penalizes clusterings that
 - always group the same pairs together **or**
 - always separate the same pairs across layers.
- Overall objective:
 - reconstruction (quality per layer & view),
 - view-weighted fusion,
 - balanced redundancy between clusterings from different layers.

Summary — Multi-View / Multi-Source Multiple Clustering

Key ideas

- Multi-view data provide **shared** and **view-specific** structure.
- Models use **view-specific encoders / subspaces** plus a **shared latent space** to balance **consensus** and **complementarity**.
- Classical DiMSC shows how to add **diversity across views** via HSIC-based regularization on subspace representations.
- DMClusts extends this to deep settings with
 - multi-layer factorization,
 - one clustering per layer,
 - view weights and balanced redundancy across clusterings.

Bridge to next part

- Multi-view methods show how to exploit **structured inputs** (views).
- The next step: bring **humans into the loop** —
 - How can user preferences, prompts, or constraints guide *which* multiple clusterings we generate?

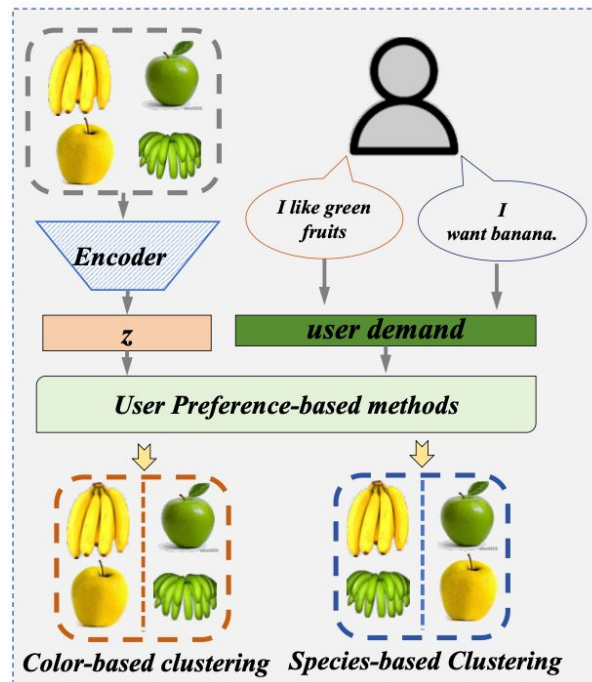
User-Guided & Interactive Multiple Clustering

Why User-Guided Multiple Clustering?

Problem: too many clusterings, not enough intent

- Deep multiple clustering methods can produce **many** valid clusterings.
- But users usually care about **one or two specific aspects** (e.g., *color-based* vs. *species-based* grouping of images).
- Without guidance, users must **inspect all clusterings** and guess which one matches their intent.

We need ways to **encode user intent** and turn it into multiple, user-aligned clusterings.



Roadmap — How Do We Inject User Intent?

Three levels of guidance we will cover

1. Hand-crafted preferences (low-level control)

- Users choose / weight **features or attributes**, or tweak distances.
- Examples: TDMC, tensor-based methods with user-chosen feature groups.

2. Semantic / augmentation-based guidance (mid-level control)

- Encode “what aspect to focus on” via **augmentations or proxy features**.
- Example: AugDMC — different augmentations → different semantic views.

3. Natural-language & multi-modal interfaces (high-level control)

- Users provide **keywords or short sentences** describing desired views.
- LLMs + CLIP turn these into **proxies / subspaces** (Multi-MaP, Multi-Sub).

Level 1 — Hand-Crafted Preferences: Tensor-Based Multiple Clustering

How user intent enters

- Users **manually select** the attributes or feature groups they care about
 - e.g., {time, temperature, wind speed} vs. {age, income, location}.
- The system builds a **similarity / distance** that emphasizes the selected attributes
 - different selections → different similarity tensors → different clusterings.

Tensor-based multiple clustering (TMC / TDMC)

- Represent multi-source data as a **high-order tensor**
 - each mode corresponds to an entity type or attribute group.
- **TMC**: factorize the tensor (e.g., Tucker / CP) to obtain several factors / subspaces; clustering in these factors yields multiple views of the data.
- **TDMC**: extend TMC so that
 - different **user-chosen attribute combinations** correspond to different tensor slices / weightings,
 - each combination induces its own customized clustering.

Level 1 — Hand-Crafted Preferences: Tensor-Based Multiple Clustering

Pros & limitations

- **Pros**
 - Very **explicit control**: users directly choose which attributes drive each clustering.
 - Naturally fits cyber–physical–social data with many structured attributes.
- **Limitations**
 - Requires users to understand features well (expert-style interface).
 - Intent can only be expressed via **feature selection / weights**, not via higher-level semantics or natural language.

Level 2 — Semantic / Augmentation-Guided Multiple Clustering (AugDMC)

Motivation

- Deep multiple clustering methods improve quality, but still struggle to **capture a user's preferred aspect** of the data.
- Observation: different data **augmentations preserve different aspects**
 - e.g., color jitter often preserves *shape* but changes *color*;
 - rotation / crop preserve *color* but change *shape*.

Level 2 — Semantic / Augmentation-Guided Multiple Clustering (AugDMC)

Core idea of AugDMC

- Use **prototype-based self-supervised learning** to learn image features

$$f_{\theta}(x_i) \in \mathbb{R}^d$$

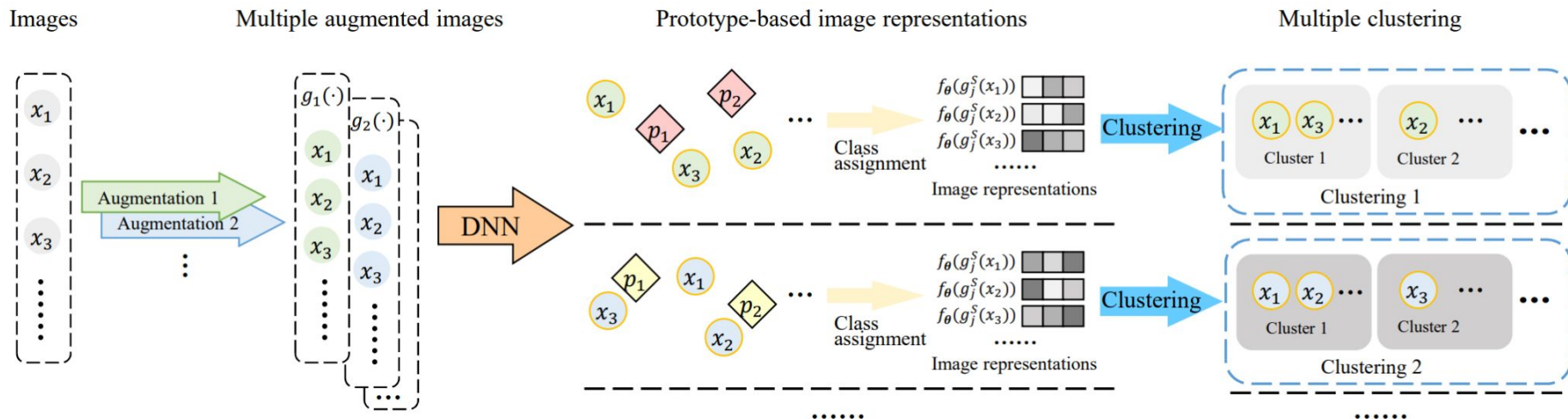
with prototypes $\{p_k\}_{k=1}^K$ as latent class anchors.

- Given a set of augmentations $\{g_1, \dots, g_J\}$:
 - For each augmentation g_j , learn features that are **invariant** to g_j :

$$\min_{\theta} \sum_i \|f_{\theta}(g_j(x_i)) - f_{\theta}(x_i)\|^2$$

- Different augmentations $\rightarrow$ **different invariant properties** $\rightarrow$ different **aspects** of the data (e.g., color vs. shape).
- For each augmentation (aspect), feed the learned features into a **single-clustering algorithm** (e.g., k-means) to obtain one clustering solution.

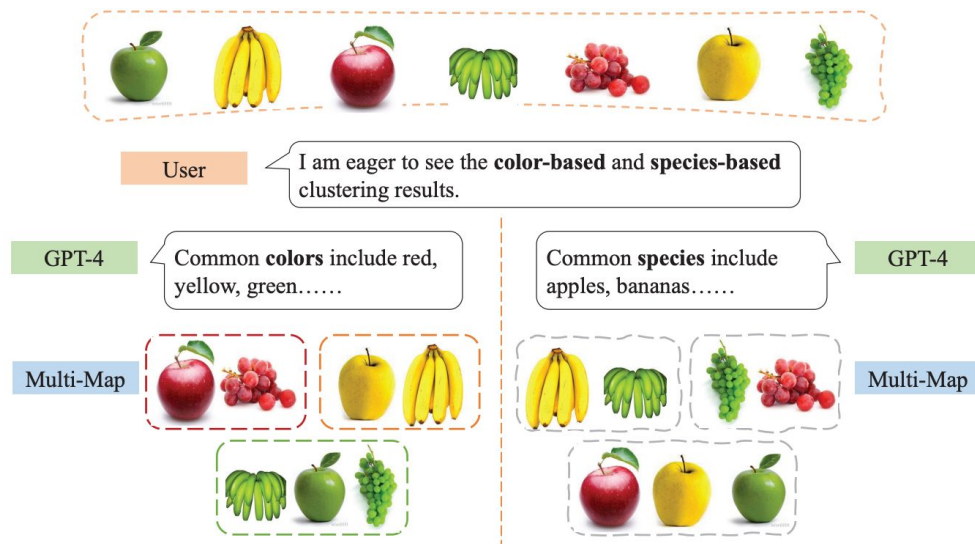
Level 2 — Semantic / Augmentation-Guided Multiple Clustering (AugDMC)



Level 3 — Natural-Language Proxies: Multi-MaP (High-Level View)

User intent & goal

- Input: an **unlabeled image set**.
- User provides a few **high-level concepts** in text, e.g.
“I want a **color-based** clustering and a **species-based** clustering.”
- Goal: turn each concept into a **separate clustering** of the same images.



*Multi-MaP: from user's high-level concept → GPT-4 reference words
→ CLIP proxies → user-aligned clusterings.*

Level 3 — Natural-Language Proxies: Multi-MaP (High-Level View)

Step 1 – From user text to reference words (GPT-4)

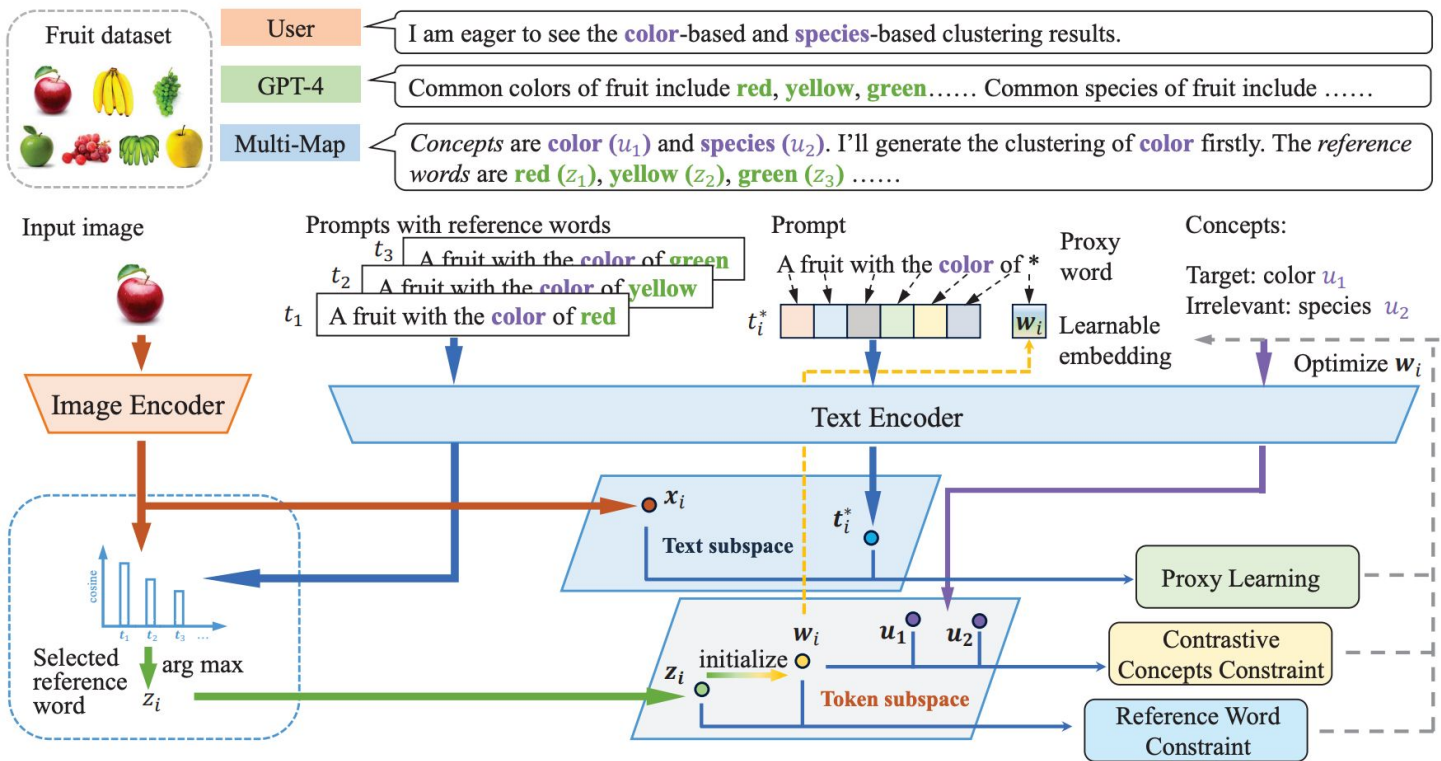
- For each user concept (e.g., “color”, “species”), GPT-4 generates:
 - a short **text description** (context), and
 - a set of **reference words** related to that concept
 - Color: {red, blue, yellow, ...}
 - Species: {apple, banana, orange, ...}
- These words live naturally in CLIP’s **text space** and anchor the concepts.

Step 2 – Multi-modal proxies with CLIP

- Use CLIP to encode **images** and **text** into a joint embedding space.
- For each concept, introduce a learnable **text proxy** (a trainable text embedding).
- Initialize each proxy near its reference words, then
 - **pull** images relevant to the concept closer to the proxy,
 - **push** contrastive images further away.

Result: for each concept, we obtain a **proxy-guided embedding** of all images.

Level 3 — Natural-Language Proxies: Multi-MaP (High-Level View)



Multi-MaP — Proxy Learning and User-Aligned Clusterings

How proxies are optimized

- For each concept-specific proxy:
 - Keep it **close to** its reference words
 - reference-word constraint (avoid drifting off the text manifold).
 - Ensure proxies for different concepts are **distinct**
 - concept-level constraint (avoid collapsing “color” and “species” into the same thing).
- Training objective (sketch):
 - similarity loss between image features and the right proxy,
 - contrastive loss against other concepts,
 - regularizers to tie proxies to their reference words.

From proxies to multiple clusterings

- After training, each concept has its own **proxy-guided embedding space**.
- On each space, apply a **standard clustering algorithm** (e.g., k-means).
 - Color proxy → color-based clustering.
 - Species proxy → species-based clustering.
 - Background proxy → background-based clustering, etc.

So the number and type of clusterings are directly **driven by the user’s concepts**.

Multi-MaP — Proxy Learning and User-Aligned Clusterings

Key contributions (conceptual)

- **User-driven specification of cluster “aspects”**
 - Users only provide a few **high-level concepts** in natural language (e.g., color, species, background), and Multi-MaP turns them into **separate clustering tasks** on the same data.
- **Multi-modal proxy learning for concepts**
 - Introduces **learnable text proxies** in CLIP space, constrained by GPT-4 reference words and concept-level separation.
 - Proxies act as **anchors** that pull image features toward the intended concept and push against contrastive concepts.
- **End-to-end path from intent → representations → multiple clusterings**
 - From user text → GPT-4 → proxies → proxy-guided embeddings → *one clustering per concept*, without requiring labels.
 - Bridges **LLMs, vision–language models, and multiple clustering** in a single framework.

Multi-Sub — Customized Multiple Clustering via Multi-Modal Subspace Proxies

Why go beyond Multi-MaP?

- Multi-MaP already turns user concepts into CLIP-based proxies, but:
 - It often needs a **contrastive concept** from the user.
 - Representation learning and clustering are mostly **two-stage**.
 - A single proxy per concept does not fully exploit the **subspace structure** of text.

Key idea of Multi-Sub

- Treat each user concept (e.g., “color”, “species”) as a **subspace** in CLIP’s text space, spanned by **reference words** from GPT-4.
- For each image and concept, learn a **proxy word** as a weighted combination of these reference words.
- Use these proxy words to define a **concept-specific multimodal subspace** and learn an image projection into that subspace.
- Cluster in each learned subspace → **one clustering per user concept**, with representation learning and clustering optimized together.

Multi-Sub — Phase I: Subspace Proxy Learning

Reference words as a concept subspace

- For a user concept (e.g., “color”), GPT-4 returns a set of **reference words** (e.g., red, yellow, green, ...).
- These words define a **text subspace** inside CLIP’s text embedding space.

From each image to a proxy word

- For each image $\mathcal{X}_i$, Multi-Sub learns a **latent factor** that assigns weights $\{a_{i,1}, \dots, a_{i,K}\}$ over the reference words (softmax over similarities).
- The **proxy word** for image i under this concept is
$$w_i = \sum_{k=1}^K a_{i,k} \phi(z_k),$$

i.e., a weighted combination of the embeddings of the reference words.

- Intuition:
 - the weights $\{a_{i,k}\}$ locate the image within the **concept subspace**,
 - w_i is a concept-specific textual description of the image.

Multi-Sub — Phase I: Subspace Proxy Learning

Aligning images and proxy words

- Build a CLIP-style prompt that inserts w_i into a template, then encode it with the (frozen) CLIP text encoder.
- Learn an image-side projection so that
 - the image embedding $f(x_i)$ is **close to** its proxy prompt embedding,
 - and **separated from** proxies of other images / concepts.

Result of Phase I:

for each concept, we obtain a **multi-modal subspace** where images and their proxy words are aligned.

Multi-Sub — Phase II: Clustering and Contributions

Phase II – Clustering in the learned subspace

- Use the aligned image embeddings (from Phase I) and proxy information as **pseudo-labels / guidance**.
- Add a **clustering loss** (e.g., k-means-style or DEC-style) on top of the concept-specific subspace:
 - encourage compact, well-separated clusters,
 - keep them consistent with the proxy structure.
- Alternate Phase I and Phase II:
 - Phase I: improve subspace proxies and image–text alignment.
 - Phase II: refine clusters and image projection under that concept.

What Multi-Sub contributes

- **Subspace-based natural-language guidance**
 - user concept → subspace spanned by reference words, not just a single proxy vector.
- **No contrastive concept required**
 - each clustering is driven by a **single** high-level concept.
- **End-to-end concept-aware clustering**

Summary — User-Guided & Interactive Multiple Clustering

From unsupervised to user-guided

- We moved from “**What clusterings exist?**” to “**Which clusterings does the user actually want?**”.

Three levels of guidance

- **Level 1 — Hand-crafted preferences (TMC / TDMC)**
 - Users pick / weight **attributes** → different similarities / tensors → different clusterings.
 - Very explicit, but low-level and expert-oriented.
- **Level 2 — Semantic / augmentation guidance (AugDMC)**
 - Designed **augmentations** encode which **aspect** to keep (shape, color, texture, ...).
 - Prototype-based self-supervision → one clustering per augmentation-defined aspect.
- **Level 3 — Natural-language proxies (Multi-MaP, Multi-Sub)**
 - Users provide **short text concepts** (color, species, background, ...).
 - GPT-4 + CLIP learn **(sub)space proxies** and concept-specific embeddings, then cluster in these spaces to get user-aligned solutions.

Evaluation, Benchmarks & Open Problems

Evaluation & Benchmarks for Multiple Clustering

Why evaluation is tricky here

- We no longer have a **single clustering** to evaluate.
- We care about:
 - quality of **each clustering**, and
 - **diversity / non-redundancy** across clusterings.
- For user-guided methods, we also care about **alignment with user intent**.

Evaluation & Benchmarks for Multiple Clustering

Three evaluation angles

- **Per-solution quality**
 - Standard clustering metrics: ACC, NMI, ARI, silhouette, etc.
 - For multi-view: quality on each view & on fused representations.
- **Diversity / redundancy between solutions**
 - Similarity between partitions: NMI, VI, adjusted mutual information.
 - Pairwise co-association matrices: are we grouping exactly the same pairs?
 - Subspace diversity: orthogonality / independence between subspaces or heads.
- **Stability & robustness**
 - How sensitive are solutions to initialization, sampling, augmentations, or noisy views?
 - Do we keep diverse, meaningful solutions under perturbations?

Benchmark Datasets for Multiple Clustering

(A) Visual multiple-clustering benchmarks (single-view / vision-focused)

- **Fruit**
 - 105 fruit images; 2 clusterings: **species** (apple, banana, grape) and **color** (green, red, yellow).
- **Fruit360**
 - 4,856 images; species (apple, banana, cherry, grape) and color (red, green, yellow, maroon).
- **Card**
 - 8,029 playing-card images; 2 clusterings: **rank** (Ace, King, ...) and **suit** (clubs, diamonds, hearts, spades).
- **ALOI**
 - Amsterdam Library of Object Images; used with 2 clusterings: **color** and **shape**.
- **CMUface**
 - 640 face images; 4 clusterings: **emotion**, **glasses**, **identity**, **pose** (multi-aspect face benchmark).
- **Flowers**
 - 1,600 flower images; 2 clusterings: **color** and **species**.
- **Stanford Cars**
 - 1,200 car images; 2 clusterings: **color** (red, blue, black, ...) and **type** (sedan, SUV, convertible, ...).
- **CIFAR-10 (multi-clustering variant)**
 - 60,000 images; 2 clusterings constructed for Multi-Sub:
 - **type**: transportation vs. animals
 - **environment**: land vs. air vs. water

Benchmark Datasets for Multiple Clustering

(B) Multi-view / multi-source benchmarks (classical + deep MVC)

(canonical seven multi-view datasets used in DMClusts)

- **Caltech7**
 - 1,474 samples, 7 classes, **6 views** (Gabor, wavelet, CENTRIST, GIST, HOG, LBP).
- **Handwritten**
 - 2,000 digit images, 10 classes, **6 views** (pixel statistics, profiles, Fourier, etc.).
- **Reuters**
 - 1,200 documents, 6 classes, **5 text views** (TF-IDF on different languages / feature spaces).
- **BBCSport**
 - 145 sports articles, 2 classes, **4 text views**.
- **MSRCv1**
 - 210 images, 7 classes, **6 visual views** (color, texture, SIFT, etc.).
- **Yale**
 - 165 face images, 15 classes, **3 views** (different feature descriptors).
- **Mirflickr**
 - 16,738 images, 24 classes, **2 views** (visual features + tag features).

Practical Evaluation Tips for Multiple Clustering Experiments

Always separate “quality” from “diversity”

- **Per-clustering quality** (within each solution)
 - Use standard clustering metrics: **ACC** / **NMI** / **ARI**, silhouette, Dunn index, etc.
 - For multi-view methods, report quality on **each view** and on fused / shared representations.
- **Between-clustering diversity / redundancy**
 - Use similarity between partitions: **NMI**, **VI**, **AMI**, **Jaccard coefficient**.
 - Co-association / overlap based measures (as in DMClusters) to penalize clusterings that always group the same pairs together or always separate them.

Tutorial Recap

1. Problem perspective

- Move from **one best clustering**
→ to **multiple high-quality, diverse partitions** of the same data.
- Formal lens:
 - per-solution **quality**, and
 - cross-solution **diversity / non-redundancy**.

2. Methodological progression

- **Classical foundations**
 - Alternative / disparate clusterings; subspace-based, spectral, and density-based approaches.
- **Deep representation-based methods**
 - Encoders + multi-head / multi-subspace designs with diversity losses (e.g., DivClust, DDMC).
- **Multi-view / multi-source methods**
 - Shared vs. view-specific structure, consensus vs. complementarity (DiMSC, DMClusters).
- **User-guided & interactive methods**
 - From feature weights (TMC/TDMC) → augmentations (AugDMC) → natural-language proxies and subspaces (Multi-MaP).

3. Evaluation & practice

- Benchmarks explicitly designed with **multiple aspects** (Fruit-family, Card, CMUface, Cars, multi-view sets).
- Evaluation separates **quality**, **diversity**, and **intent alignment**, rather than reporting a single clustering score.

Open Problems and Future Directions

Beyond clean benchmarks

- **Imperfect, large-scale data**
 - Heterogeneous, missing, or noisy views; streaming and continual settings.
 - Need scalable, incremental multiple clustering with bounded drift.
- **Robustness & stability**
 - How stable are multiple clusterings under perturbations, re-sampling, or prompt changes?
 - Formal notions of *stable yet diverse* solution sets are still underdeveloped.

Open Problems and Future Directions

Human–model collaboration

- **Intent modelling**
 - Interpreting ambiguous or conflicting user instructions; updating clusterings as user goals evolve over time.
- **Interactive interfaces**
 - Designing tools where non-experts can safely steer clusterings without needing to understand features, losses, or architectures.

Responsible use

- **Fairness & ethics**
 - Multiple clusterings can expose sensitive structure; need fairness-aware objectives and auditing protocols.
- **Privacy & security**
 - How to support multiple clustering under privacy constraints (federated settings, differential privacy, restricted access data)?

A Landscape of Multiple Clustering Methods

Method & Reference	Category	Input Modality				Advantages	Limitations
		Images	Text	Audio	Tabular		
DiMSC [Cao <i>et al.</i> , 2015]	B	✓				Reduced redundancy	Limited scalability
TMC [Zhao <i>et al.</i> , 2017]	A				✓	Flexible weighting	High cost
LMSC [Zhang <i>et al.</i> , 2017]	B	✓				Robust fusion	Complex optimization
TDMC [Zhao <i>et al.</i> , 2018b]	C				✓	Customizable results	User-dependent
MISC [Wang <i>et al.</i> , 2019]	A	✓				Independent subspace	Two-stage complexity
DAMC [Li <i>et al.</i> , 2019b]	A	✓				Adversarial diversity	Training instability
ENRC [Miklautz <i>et al.</i> , 2020]	A	✓				Non-redundant embeddings	Dimension tuning
DEMVC [Xu <i>et al.</i> , 2021]	B	✓				Coordinated centers	High complexity
iMClusts [Ren <i>et al.</i> , 2022]	A	✓				Attention-based	Interpretability gap
MFLVC [Xu <i>et al.</i> , 2022b]	B	✓	✓	✓		Multi-level alignment	Parameter sensitive
CMHHC [Lin <i>et al.</i> , 2022]	B	✓				Contrastive alignment	Negative sampling bias
scMCs [Ren <i>et al.</i> , 2023]	B				✓	Complementary subspaces	Restricted generalization
AugDMC [Yao <i>et al.</i> , 2023]	C	✓				Automated multi-solution	Augmentation-sensitive
Xiong et al. [Xiong <i>et al.</i> , 2024b]	B	✓	✓			Anchor-based scaling	Sparse supervision
Fu et al. [Fu <i>et al.</i> , 2024]	B	✓	✓			Subspace-contrastive	Extra network design
DDMC [Yao and Hu, 2024]	A	✓				Disentangled factors	Variational complexity
Multi-MaP [Yao <i>et al.</i> , 2024b]	C	✓	✓			User-oriented	Prompt ambiguity
Multi-Sub [Yao <i>et al.</i> , 2024a]	C	✓	✓			Proxy-based flexibility	Additional overhead

Comparison of representative deep multiple clustering methods in chronological order. (A) Deep Representation-based; (B) Multi-view/Multi-source; (C) User Preference-based.

Thank You!